

■ React State Management

Key Concepts – Easy English Guide

1. Update State Based on Old State

In React, when you want to update state using its **current value**, always pass a function to the state setter. This makes sure you are always working with the most recent value.

- Use a **callback function** inside `setState()` to read the previous state safely.
- React may batch multiple updates, so using the old value directly can sometimes give wrong results.

```
setCount(prev => prev + 1) // ■ Safe way
```

■ *Always use the function form when the new state depends on the old state.*

2. Two-Way Data Binding (Function Component)

Two-way data binding means the **UI shows the state** and the **state updates when UI changes**. In function components, this is done with `useState` and an `onChange` handler.

- Bind the input value to a state variable.
- Use `onChange` to update state whenever the user types.

```
<input value={name} onChange={e => setName(e.target.value)} />
```

■ *This keeps your UI and data always in sync with each other.*

3. Updating State with Spread Operator

React state must **never be changed directly**. When your state is an object or array, use the **spread operator** (`...`) to create a new copy with your updates.

- Spread copies all existing fields, then you override only what changed.
- Mutating state directly will not trigger a re-render.

```
setUser(prev => ({ ...prev, age: 25 })) // ■ Correct
```

```
user.age = 25 // ■ Never do this
```

■ *Always return a new object/array – never mutate the existing one.*

4. Lifting State Up

When a **child component** needs to tell the **parent** about something (like a button click), the parent passes a **function as a prop**. The child calls that function — this is called lifting state up.

- **Sharing State Across Components:** Move state to the closest common parent so both siblings can access it.
- **Lifting Computed Values Up:** If multiple children need the same calculated result, compute it in the parent and pass it down as a prop.

```
// Parent passes handler to child <Child onSelect={handleSelect} /> // Child calls it props.onSelect(selectedItem)
```

■ *Think of the parent as the manager — children report up, parent decides what to do.*

5. Avoid Intersecting States

Don't keep two state variables that represent the same thing. When states overlap or depend on each other, bugs happen easily.

- **Prefer Computed Values:** If a value can be calculated from existing state or props, just calculate it — don't store it as state.
- **Avoid Unnecessary State Management:** Ask yourself – can I *derive* this instead of storing it?

```
// ■ Storing redundant state const [items, setItems] = useState([]); const [count, setCount] = useState(0); // count === items.length! // ■ Compute instead const count = items.length;
```

■ *Less state = fewer bugs and simpler code.*

6. Deriving State From Props

Sometimes the initial value of state comes from a **prop**. Be careful — if the prop changes later, the state won't update automatically (state is only set once from the prop at mount).

- Use props directly in rendering if the value is always in sync with props.
- Only use props to *initialize* state when you want the component to manage it independently after that.

■ *Prefer using props directly over copying them to state, unless you need local control.*

7. Reducing State & Identifying Unnecessary State

Before adding a new state variable, always ask: *Do I really need this as state?* Unnecessary state makes components harder to understand.

- Is it already in props? → Use the prop.
- Can I calculate it from existing state or props? → Derive it.
- Only store state when the component itself needs to remember something over time.

■ *Simpler state = simpler component = easier to debug.*

8. Deriving Computed Values From Other Computed Values

You can chain calculations — a computed value can be built from another computed value. This keeps your logic clean and your state minimal.

```
const total = items.reduce((s, i) => s + i.price, 0); const tax = total * 0.18;  
const grandTotal = total + tax;
```

- None of tax or grandTotal need to be stored as state.
- They will always be recalculated correctly when items change.

■ *Chain your computations – they are free and always accurate.*

9. Why Immutability Matters – Always!

React works by **comparing old and new state** to decide what to re-render. If you change (mutate) state directly, React cannot detect the change — so your UI won't update.

- Always create a **new copy** of objects and arrays.
- Use ... spread, .map(), .filter(), or .concat() to make copies.
- This also prevents hard-to-find bugs where old references unexpectedly change.

```
// ■ Immutable update setList(prev => [...prev, newItem]); // ■ Mutating directly  
list.push(newItem); // React won't see this!
```

■ *Treat state as read-only. Always make a new copy before changing.*

10. An Alternative to Lifting State Up

Lifting state up is great, but when your app grows large, it can get messy passing props through many levels (**prop drilling**). There are cleaner alternatives:

- **React Context API:** Share state across many components without passing props manually at every level.
- **useReducer:** Great for complex state logic – keeps update rules in one place.
- **State management libraries** (like Zustand, Redux): For very large apps with shared global state.

■ *Start with lifting state, switch to Context when prop drilling gets painful.*

■ Code Examples

<https://github.com/academind/react-complete-guide-course-resources/tree/main/code/04%20Essentials%20Deep%20Dive/>